Computer Graphics: Lecture 10

Animation, culling, and Depth-buffers

Welcome back!

Last time:

- We learned about affine matrices.
- We learned about the homogenous coordinates needed to make use of them. Our points are now vectors with a 1 in the w-place.
- We learned how to send matrix data to the GPU.
- We stretched and squashed some triangles

This time

We have one main objective: animate a spinning figure.

In order to accomplish this cleanly, we will have to examine two more techniques:

1. Back-face culling
2. Depth buffering (AKA the Z-buffer)

Animation

Let's stop having all the power of our GPU used to render one frame.

It's meant to be able to generate dozens or even hundreds of frames every second, so lets learn how to do that.

It's slightly more involved than just drawing a bunch of pictures, though.

In particular: [how many pictures do we need to draw?]

Frame rates

We call the number of images we can draw in a certain amount of time the **frame rate**.

Frame rate is measured most commonly in **Frames Per Second (FPS)**.

In computer graphics, given that we often want to optimize to reduce the time it takes to render one frame, we also like to use **MilliSeconds Per Frame (msPF)** too.

The reciprocal is nice, because an increase of 30 FPS is meaningless if we already are at 1000 FPS, but huge if we're at 10 FPS. But reducing by 1 millisecond per frame always means the same level of improvement.

Frame rates (2)

The typical human eyes and brain can perceive up to 12 images per second as distinct images.

Once the number goes higher than that, we start to perceive it as motion.

We can still pick out variations in luminance at extremely high framerates, however (Source). Up to 500 Hz.

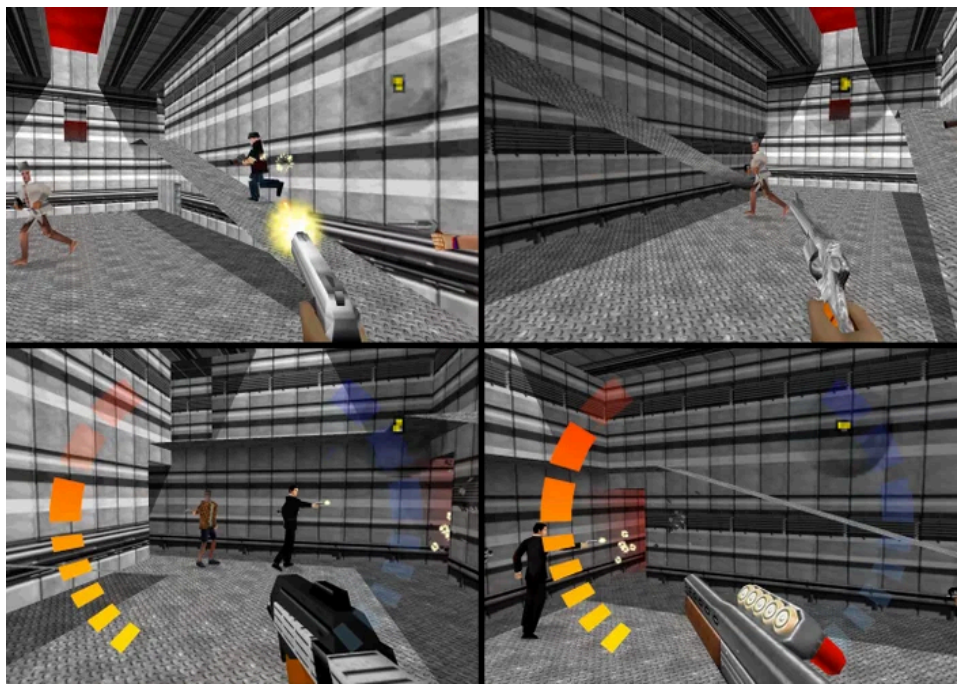
However, for motion pictures, 24 frames per second is widely used and generally considered fine.

Frame rates in interactive software

So if 24 FPS is fine, why are there gamer monitors out there that advertise a refresh rate of 240 Hz.?

Afterall, famously influential shooter Goldeneye 007 barely managed hit 30 FPS some of the time, and contemporary critics didn't seem to mind.

[thoughts?]



This game in this scene was likely struggling to hit even the 12 FPS required for motion perception.

Games are different

The reason is that games are interactive.

The frame rate doesn't just determine how smooth the animation is, it also determines how long you have to wait until seeing the result of your actions.

This turns out to have a large impact on how skilled people are (source). More frames means more chances to aim adjust.

Subjectively, I can also say that higher frame rates just make games more fun in general.

What frame rate to target?

So, logically, that means we want to render as many frames as possible…

Except, your monitor is only capable of displaying a certain frame rate.

If your monitor only supports 120 Hz., displaying more is not only pointless, it can actually hurt the image quality.

The issue is that the monitor draws the frame from top to bottom. If you draw too quickly, you can actually replace the frame its drawing *as it's being drawn*, causing a visible discontinuity known as **tearing**.

Tearing

Notice the seam about a quarter of the way down the image.

Questions?

Separating setup vs rendering

Previously, when we were drawing one frame, we didn't really differentiate between the setup code and the code that actually drew the frame.

Setup

- Compiling the shader module
- Creating the buffers, bind groups, and textures
- Creating the pipeline

Actual rendering

- Creating the encoder
- Building and submitting the pass

Things that have to happen every frame

Creating the command encoder has to happen every frame. It is illegal to reuse an already `.finish()`ed command encoder.

Therefore, at the very least, the code where we define our render pass and submit the commands to the device's queue has to happen once per frame. We will be doing this often.

Things that only have to happen once

On the other hand, if something only has to happen once, it is wastful to do it every frame.

Compiling the shader is slow, and it would be ridiculous to do it often.

Same with creating buffers. It's slow and involves allocating memory.

Same with textures.

In general, we want to do these things once, then reuse them.

Creating textures and geometry (vertex data) usually happens once while a game is *loading*. Once it's loaded, the frames are fast to draw.

Setting this up

From now on, our samples are going to be **classes**.

A class in TypeScript works similarly to classes in other languages you are familiar with, such as Java and Python.

When we have a class, we can use the `new` keyword to create an instance of it. The instance will have a copy of all the fields of the class, and we can invoke methods on it.

```
const myInstance = new MyClass(dataTheClassNeeds);
```

`myInstance` is now an object, and its type is `MyClass`.

Fields

In order for an object to be useful, it needs to have data inside.[1]

The definitions of what kinds of data an object will have are **fields**.

Defining fields in TypeScript looks just like defining parameters:

```
class Foo{
  someField: number = 0; // default value of 0.
}
// now we can make a Foo and access the field
const f = new Foo(); console.log(f.someField);
```

[1] If an object does not have data, then you're just bundling a bunch of functions together. This is fine, but the object is just a module.

Methods

The main purpose of classes is to bundle **state** (the fields of an instance) with special functions that modify that state.

These functions are called **methods**.

To make a method in typescript, we have the name of the method, followed by a parameter list and return type. **There is no special keyword for creating methods**.

```
class Foo {
  someField: number = 0;
  incField(): void { this.someField++; } // make it bigger
}
```

The `this` keyword

To access the instance that the method was invoked on, use `this`.

Suppose we did this:

```
const f = new Foo();
f.incField();
```

Inside the `incField` method, `this` would refer to `f`, since that is the instance the method is being invoked on.

We are going to take the `someField` field inside that instance, and increment it by one.

Constructors

The purpose of the constructor is to run after memory has been allocated for an object in order to initialize it to a known state.

TypeScript is very picky about constructors. *Every field* must be fully initialized to an instance of its type by the constructor.

```
class Foo {
  someField: number;
  constructor() {
    this.someField = 7; //*required*
  }
}
```

Constructors (2)

The constructor is a special method named `constructor`.

It is called after `new` is invoked. The arguments it receives are the arguments that are passed to `new`, and they must match.

If we declared `someField: number | undefined`, we could skip defining it, because every field starts out `undefined` by default, and we're saying "it can be a number or it can be undefined."

But `undefined` values are annoying, so we want to avoid them.

Default values are assigned before the constructor runs, and the constructor does not have to overwrite them.

That's it for OO

That's the only OO we're going to cover, but it should be enough to start making our samples as classes.

This will be convenient, as it will let us put the setup code that needs to happen in the constructor or in a function that runs before the constructor (like for loading textures).

Then, we can call render many times and it won't have to re-compute everything.

Let's take questions, then draw a single animated quad.

Questions?

Let's animate a quad: the class

We'll start by defining all the data we want to create up front:

```
class Sample07 {
    device: GPUDevice;
    context: GPUCanvasContext;
    pipeline: GPURenderPipeline;
    matBuf: GPUBuffer;
    vertBuf: GPUBuffer;
    textureFormat: GPUTextureFormat;
    bindGroup: GPUBindGroup;

    rotationTurns: number;
    lastRenderTime: number;
```

The class (2)

Here, all the data we want to reuse or have access to when rendering is made into a field.

We'll receive the device and context from the caller. We'll initialize the pipeline, buffers, and bind groups in the constructor.

The last two fields are needed to keep track of our animation. We're going to rotate the model around smoothly, so we want to keep track of how rotated it is (in turns).

Lastly, keeping track of the last render time lets us compute how much more we need to rotate the model.

The constructor

Inside our class, we have a constructor. It's the constructor's job to make sure to initialize all those fields:

```
constructor(device: GPUDevice, context: GPUCanvasContext) {
  this.device = device;
  this.context = context;
  this.rotationTurns = 0;
  this.textureFormat =
    (context.getCurrentTexture().format + '-srgb') as
            GPUTextureFormat;
  this.lastRenderTime = performance.now();
```

Let's quickly discuss `performance.now()`...

The performance timer

Most games and simulations need more precise timing than the basic timer the operating system and web browser use.

For these special applications, modern operating systems have a special timer called a “high performance timer”

This timer effectly tracks the number of CPU clock cycles that have passed since a program started.

`performance` is an instance of this timer which every web page has access to. `performance.now()` returns the number of milliseconds since the webpage started to be loaded. Every time we render, we’ll take the difference between `now` and `lastRenderTime` to get the delta.

Let's animate a quad: the vertices

We are going to locate it a bit offset in space, because it's eventually going to be the side of a cube that is centered at the origin:

```
const vertArray = new Float32Array([
    -.3, -.3, -.3,  1, 0, 0, // offset z by -0.3
     .3, -.3, -.3,  1, 0, 0,
     .3,  .3, -.3,  1, 0, 0,
    -.3, -.3, -.3,  1, 0, 0,
     .3,  .3, -.3,  1, 0, 0,
    -.3,  .3, -.3,  1, 0, 0,
]); // we're defining XYZ position and RGB color
```

Creating the buffers

Creating a buffer is pretty verbose. We've already seen how to do it for vertex buffers and for matrices.

However, *these* matrices are a different: they will change every frame.

Therfore, when we create the buffer for our matrix, we're going to hardcode its size, and have `mappedAtCreation` be false.

```
this.matBuf = device.createBuffer({
    size: 16 * 4,
    usage: GPUBufferUsage.UNIFORM | GPUBufferUsage.COPY_DST,
    label: "matrix buffer",
    mappedAtCreation: false,
});
```

Creating the buffers (2)

The `COPY_DST` usage is important because we are going to use a different mechanism to load our matrix buffer.

We are not going to map it and then `set` the data. Instead, we are going to use `device.queue.writeBuffer`. This function will overwrite buffer data on the GPU, but it requires the buffer support `COPY_DST`.

We'll show how to use that method in a bit. For now, let's continue discussing the constructor.

(all of this can be found, with some additional code we will discuss in the lecture, in `sample07`. I'm leaving a lot out because it can't all possibly fit in the lecture slides)

Creating the bind group layout

```
const bgLayout = device.createBindGroupLayout({
    entries: [{
        binding: 0, // the only binding is one matrix
        visibility: GPUShaderStage.VERTEX,
        buffer: {}
    }]
});
```

Our pipeline needs to know what bind groups can be bound to it, so we create the layout.

This isn't a field because we can retrieve it out of the pipeline later if we need to, so there's no need to store a separate copy of it.

The shader

Let's discuss the shader these bind groups will be getting attached to.

```
const shaderCode = /*wgsl*/`
@group(0) @binding(0)
var<uniform> model: mat4x4<f32>;

struct VertexOutput {
    @builtin(position) pos: vec4f,
    @location(0) color: vec4f,
};
```

There will be one uniform variable: the model matrix. It will be a 4-by-4 matrix just like we learned. This is why our bind group has one binding, and why there is only one bind group.

The shader (2)

```
@vertex fn vs(
    @location(0) pos: vec3f,
    @location(1) color: vec3f
) -> VertexOutput
{
    var vo: VertexOutput;
    vo.pos = model * vec4f(pos, 1.0);
    vo.color = vec4f(color, 1.0);
    return vo;
}
```

The vertex shader is going to take that model matrix and multiply it by every point it receives. So if we pass a rotation shader that rotates an amount determined by the current time, we can smoothly rotate.

The shader (3)

```
@fragment fn fs(vo: VertexOutput) -> @location(0) vec4f {
    return vo.color;
}
`;
```

Lastly, we just return the color inside the vertex output.

The `@builtin(position)` value inside the vertex output is used to determine where to draw the pixels. This happens implicitly in between the vertex shader and rasterization steps of the render pipeline.

Creating the pipeline

Speaking of the pipeline…

```
this.pipeline = device.createRenderPipeline({
    layout: device.createPipelineLayout({
        bindGroupLayouts: [bgLayout],
    }),
    ...
```

Make sure it knows about our bind group layouts.

The fragment and vertex shader are (for now) identical to how they have been.

Creating the bind group

The last step in the constructor: create the bind group

```
this.bindGroup = device.createBindGroup({
    entries: [{
        binding: 0,
        resource: this.matBuf,
    }],
    layout: bgLayout
});
```

Any questions about what goes in the constructor? We're initializing all the values that only need to be set once so we don't have to do it each frame.

Questions?

Updating the matrix

Now things are going to change a little.

We're going to define a function that will be called whenever we need to determine the model's position:

```
update(dt: number): void {
    const model = mat4.create();
    mat4.translate(model, model, vec3.fromValues(0, 0, .5));
    mat4.rotateX(model, model, -.05 * TAU); // left handed: negate amount
    mat4.rotateY(model, model, -this.rotationTurns * TAU);
    this.rotationTurns += dt * TURNS_PER_SEC;
    this.rotationTurns %= 1; // wrap around once we hit 1 turn
    this.device.queue.writeBuffer(
      this.matBuf, 0, new Float32Array(model));
}
```

Updating the matrix (2)

The first thing we do is create a new matrix. This matrix will *first*, rotate around the y axis based on the amount of rotation we've added so far.

Then it will rotate by a fixed offset forward around X. (We flip the sign because we're using a right-handed matrix in a left-handed system)

Then it will translate the object forward (we're still using left-handed coordinates, so .5 means .5 into the screen).

If we didn't translate forward, the model might be cut off, since we defined it in the negative Z position. Remember that it needs to be between 0 and 1 to show up at all.

Also remember: the transformations are in *reverse order*. `rotateY` is first!

Updating the matrix (3)

Then we use `device.queue.writeBuffer`

This method takes a buffer, which in our case is the one we're using for our matrix. It also takes an offset: which for us is 0 here (we want to write over the whole thing, not skip any bytes).

Finally, it takes an array of data to copy.

We give it the new matrix we just calculated. It will overwrite the old matrix. The next time our shader runs, it will be using the updated matrix.

Updating the matrix (4)

That TURNS_PER_SEC constant is up to you. I'm using 0.25, so it turns slowly enough for me to see that it's working smoothly.

In Javascript we can use the mod operators `%` and `%=` on floats. Since 1.5 turns is equivalent to 0.5 turns, we use `%= 1` to keep only the fractional part of the float.

Javascript numbers are 64-bit doubles, so we don't really have to do this. However, if we used 32-bit floats, we would run out of precision after an hour or two if we didn't. It's actually a common bug in even modern games to keep adding to a float without wrapping it until the addition starts underflowing.

Bind groups

You might wonder why we don't need to create a new bind group. Why can we reuse the old one?

The bind group stores the mapping between uniforms and the location of the data, but it doesn't store the data itself.

We *could* create another bind group. But we'd need to allocate a new matrix, which would be slower than just overwriting the old one.

Finally, the `writeBuffer` method requires that the `COPY_DST` usage be enabled on the buffer. If we didn't do this, we would have to map the buffer. Aquiring a map after creation is inconvenient and requires async code.

What about animation

I still haven't discussed how to animate things. Luckily, the API for it is pretty simple (at least, compared with everything else we've been doing)

There is a function you can call called `requestAnimationFrame(r)`.

This function itself takes a function, `r`, which is a render function.

We use this function to alert the browser that we want it to draw something to the screen when it's ready.

When the browser is ready to paint the screen, it will call the function you gave it as a **callback**.

Callbacks

A **callback** is a function that is called by a different system than the one which defined it.

We define a function called `render`, and we can pass that function to `requestAnimationFrame()`.

However, that will only work once.

To run it over and over again, at the end of our function, we'll have it re-queue itself by calling `requestAnimationFrame` again.

startRendering

```
startRendering(): void {
    const renderAndQueue = (now: number) => {
        this.render(now);
        requestAnimationFrame(renderAndQueue);
    }
    renderAndQueue(performance.now());
}
```

Here, we define a local function using anonymous function notation. `const renderAndQueue = (...) => { ... }` is mostly equivalent to `function renderAndQueue(...) { ... }` except it only lives inside the current scope

`startRendering` (2)

Inside that function, we call render, and then we call `requestAnimationFrame` on the anonymous function.

This will loop over and over: render the frame, then ask to be rendered again in the future.

Our function `renderAndQueue` takes a `now` variable. `requestAnimationFrame` will automatically pass the current performance timer to it when it calls it. The first time we call it ourselves with the performance time.

`requestAnimationFrame`

Usually `requestAnimationFrame` will wait until the monitor is ready to refresh, which means if we always request as soon as the old frame is rendered, we'll end up matching the framerate of the monitor.

It also will pause automatically if the user navigates away from the tab, which is a nice battery saver (it would be very annoying if a game continued to burn power if the user had it in what they thought was an inactive tab).

The result

Check out `lecture10/screens/rotating_quad.mp4` to see an example of what you should see if you're following along so far.

It's just a spinning quad, that is rotated according to the matrices we provided, and offset from the center of the screen.

Note: the actual sample draws a whole cube, so you will need to substitute just the vertex buffer from the slides.

Questions so far?

Adding the rest of the cube

Well shoot, now that we have animation, we can draw an *entire cube* instead of just a quad. We can actually see a spinning 3D figure instead of just a 2D one!

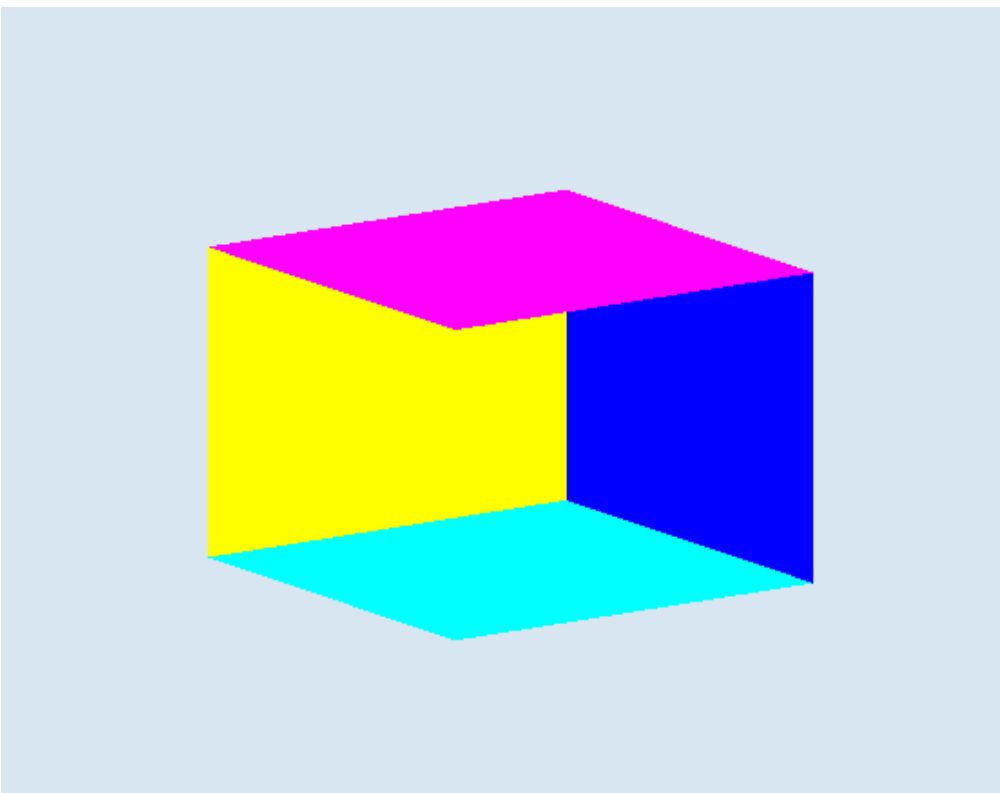
Check the sample code to see me actually specify 12 whole triangles (2 per side, 6 sides of a cube), but I'll save you the pain.

Let's see what it looks it. I will open up: `lecture10/screens/no-culling.mp4`.

Uh oh, that's weird. What's going on?

What's going on

Yeah so it's not supposed to look like this:



What's going on (2)

The problem is that web GPU is drawing the triangles in an arbitrary order. *They are not being sorted by distance.*

It just so happens that at some rotations, it looks okay, but at other rotations, we are drawing the back sides on top of the front sides in a weird way.

This completely spoils the 3D effect and looks bad.

So what do we do? Do we have to break our cube up into pieces to order the draw calls? I hope not, that would be way too slow.

How to fix it

This has been a problem as long as 3D graphics has existed, so it is not surprising that there is a simple solution that was available even on really early hardware.

I'm not talking about the depth buffer (yet), I'm talking about something even simpler: **backface culling**.

A **backface** is a face (triangle) that is pointing *away* from us.

We want to **cull** it, which means prevent it from being drawn.

Why do we want to do that?

Face Culling

Face culling serves multiple purposes:

- It makes it so that the back of a solid object does not draw over its front.
- If an object is solid, it's pointless to draw the triangles that face away from us because they will be hidden from view anyway, so it's a nice optimization even if we had some other way of blocking them.
- It ends up being required for correct transparent rendering.

So, how does the system know whether a face is facing towards us or facing away from us?

How face culling works

Simply put: the cross product between two of the face's edges.

By default, if that cross product is facing out of the screen, then the face is **front facing**. Otherwise, it is **back facing** (if it's perfectly sideways it also does not produce visible fragments).

This means we must always define our triangles in the same order: either clockwise or counter-clockwise.

If we mix the order, we won't be able to keep track of whether the cross product is positive or not.

Winding order

The order in which triangles are specified is called **winding order**.

It will determine the sign of the signed area (from the cross product).

If the winding order flips (e.g., because we rotated the triangle), the cross product will change direction too. Front-facing -> back-facing or vice versa.

The standard winding order for most 3D systems and file formats is counter-clockwise. Therefore, from now on, let's stick to counter-clockwise winding!

But how do we tell WebGPU to use culling?

Enabling culling

Culling is enabled in the pipeline. There's an optional field called `primitive` that refers to how triangles are specified:

```
primitive: { // I put this after fragment: {...}
    cullMode: 'back', // in the pipeline
    frontFace: 'ccw',
    topology: 'triangle-list',
},
```

`cullMode` can be 'back', 'front', or 'none'. 'none' means don't do culling at all. 'back' means don't draw triangles facing away from the screen. 'front' means don't draw triangles facing towards the screen (normally only used for transparent blending).

Enabling culling (2)

`frontFace` allows us to determine whether we will be using clockwise or counter-clockwise winding.

`ccw` means counter-clockwise, `cw` is clockwise.

As I said before, `ccw` is effectively standard now. But if you're working with some old data that was specified in clockwise order, you have the ability to change it.

Lastly, `topology` tells it how we will issue triangle data. `triangle-list` is the one we've been using, where we give it groups of 3 vertices. There are others that are quite useful; we'll discuss them soon.

It works

Check `lecture10/screens/with-culling.mp4` to see an example of the working quad.

Looks good!

But…it's still not quite done. Let's try drawing one more box to see why.

Drawing multiple boxes

Once we call `.setVertexBuffer` in our pass encoder, it stays until we change it. This lets us draw the same 3D model multiple times.

We can make another bind group to represent another object's model position.

```
...
pass.setVertexBuffer(0, this.vertBuf);
pass.setBindGroup(0, this.bindGroup1);
pass.draw(36); // draw the first cube
pass.setBindGroup(0, this.bindGroup2);
pass.draw(36); // draw a cube with a different matrix
...
```

Making the matrix

Let's set up a smaller cube's matrix:

```
this.matBuf2 = device.createBuffer({
    size: 16 * 4,
    usage: GPUBufferUsage.UNIFORM,
    label: "stationary matrix buffer",
    mappedAtCreation: true,
});
const stationary = mat4.create();
mat4.translate(stationary, stationary, vec3.fromValues(.4, 0, .86));
mat4.scale(stationary, stationary, vec3.fromValues(0.4, 0.4, 0.4));
mat4.rotateX(stationary, stationary, -.05 * TAU);
mat4.rotateY(stationary, stationary, -.15 * TAU);
(new Float32Array(this.matBuf2.getMappedRange())).set(stationary);
this.matBuf2.unmap();
```

This one is stationary, so we map it like normal. (no `writeBuffer`)

Making the bind group

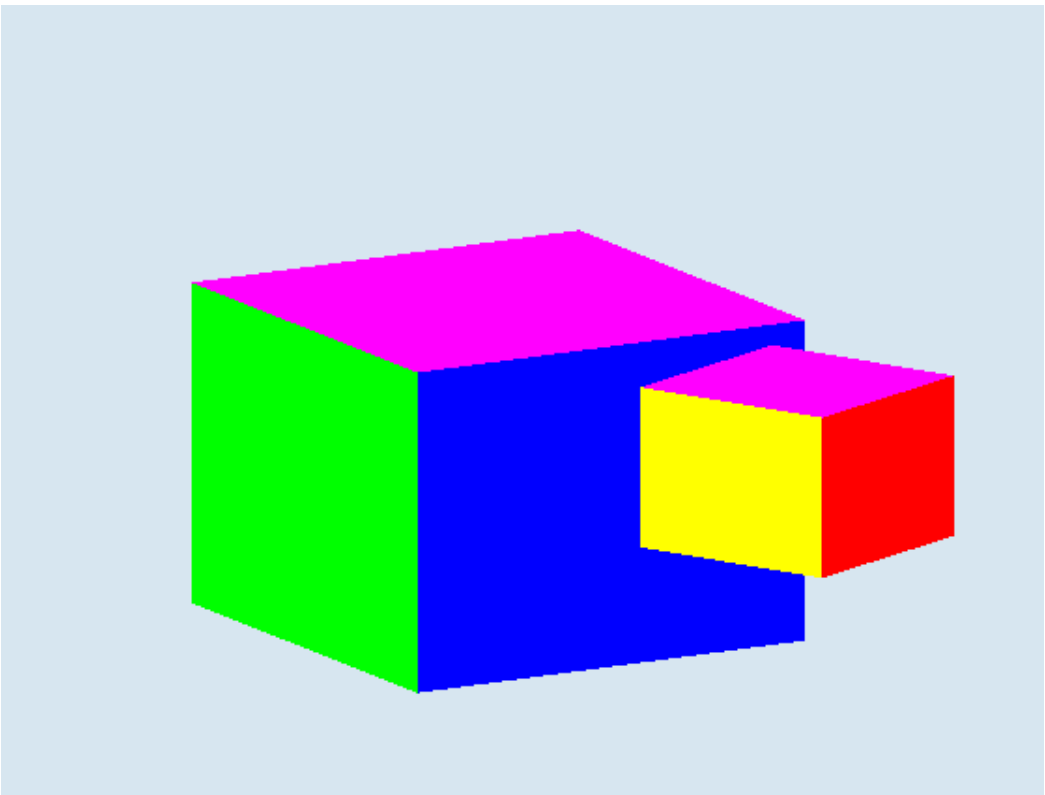
We make a second bind group for the second cube.

In general, every distinct object in your scene will have at least one bind group for it (maybe more than one).

```
this.bindGroup2 = device.createBindGroup({
    entries: [{binding: 0, resource: this.matBuf2}],
    layout: bgLayout
});
```

It's the same as the first bind group except it's pointing at the second matrix buffer.

Let's draw!



Uhh…that's still not right…

Why is it on top

We're using left-handed coordinates, so the second cube should be *behind* the first cube.

Its being translated by 0.75 into the screen, but the other cube is only being translated 0.5.

However, it's being drawn *second*. Because we drew the smaller cube second, its pixels will be copied over the first cube's.

So…just switch the order, right? Not so fast: that might work now, but what if we made the smaller cube rotate? Then it would be wrong once its position relative to the first cube changed.

The old days

In the old days, there was no solution to this other than to sort the objects in your scene so that you drew them in the right order.

This is why old 3D games on systems without a z-buffer (like the Playstation 1) are often so “grid based”. They used the grid to ensure correct drawing order.

Notice how 'grid-based' the landscape is.

The old days (2)

The Playstation actually had the ability to put polygons in buckets when submitting them for drawing. You could give it a drawing order.

Unfortunately, this isn't enough. You can define 3 triangles that all overlap each other. This is called **cyclic overlap**.

The only way to really solve the problem is to track each pixel individually, which was very expensive back in the 90s.

But it's not the old days!

The real fix here is to use some hardware that GPUs have had for decades now: the depth buffer (also called the z-buffer).

The depth buffer keeps track of, for each pixel on the screen, the *nearest* depth that we have written to that pixel.

When we draw a fragment, if its depth is *greater* than the *nearest* depth, then it must be *behind* something we have already drawn.

Therefore, it's going to be hidden.[1]

[1] I'm leaving out transparency. We still use the depth-buffer with transparency, but it's more complicated. For now, assume everything in the scene is solid.

Enabling the depth buffer

There are several things we need to do to enable depth buffering:

1. Create the depth buffer. It's a texture.
2. Put a depth buffer description in our pipeline, so it knows that a depth buffer will be there.
3. Add a depth buffer attachment to our pass, pointing to the depth buffer we created. Now the buffer will be written to and read from automatically when we draw our passes.

Depth buffer storage

A depth buffer is just a texture with a special format.

Most depth buffers are 24 bits wide. This ends up being plenty for most applications. It's basically just a 24-bit number, where larger means farther away.

The GPU will handle converting the z value of the fragment into a depth value that fits into that number.

Depth buffer storage (2)

There is one other kind of buffer that serves a similar function to the depth buffer. It's called the **stencil buffer**.

It's basically a way to control what parts of the image we want to draw to. For example, we can use the stencil buffer to protect part of an image to avoid copying over the pixels that are already there.

This is useful for a bunch of random things, like drawing a boat in the water without the water being drawn over the inside of the boat.

We won't be using the stencil buffer for your assignments, but I'm telling you about it because the storage for the depth buffer overlaps the stencil buffer, so I want you to know the term.

Creating the depth buffer

To create the depth buffer, we create a texture with the right format.

```
this.depthBuffer = device.createTexture({
    format: 'depth24plus-stencil8',
    size: {
        width: context.canvas.width,
        height: context.canvas.height,
        depthOrArrayLayers: 1
    },
    usage: GPUTextureUsage.RENDER_ATTACHMENT,
    label: 'depth buffer',
});
```

Creating the depth buffer (2)

The main thing to explain here is the format.

`depth24plus-stencil8` means "use at least 24-bits for the depth buffer, but feel free to use more. Also, give me an 8-bit stencil buffer."

We won't be using a stencil buffer for a while, but it's common for them to share space. For example, 24-bits of depth and 8-bits of stencil is a common format to pack the depth and stencil buffer into the same texture.

We don't have to worry about this. The GPU handles it for us. By using `depth24plus` instead of `depth24` we give the driver the option to use more space if it wants.

Defining the pipeline

Let's add a `depthStencil` description to our pipeline. Depth buffering and stencil buffers go together a lot, because they are both used to reject fragments.

```
depthStencil: {
    format: 'depth24plus-stencil8',
    depthCompare: 'less-equal',
    depthWriteEnabled: true,
},
```

Defining the pipeline (2)

We specify the format so that it matches our texture.

We also make sure `depthWriteEnabled` is true, so that the pipeline knows it can actually write into the depth buffer, which is the main point: we want to store the nearest pixel depth for each pixel. We only disable this typically when doing transparent rendering.

Lastly, `depthCompare` tells it what rule needs to be satisfied for the pixel to be written. `less-equal` means that the pixel needs to have a depth value less than or equal to the one in the depth buffer.

Note: there are stencil fields we're ignoring because we aren't using the stencil buffer yet.

The attachment

Lastly, after defining the `colorAttachment`, we also define a `depthStencilAttachment` in the pass description:

```
depthStencilAttachment: {
    view: this.depthBuffer,
    depthClearValue: 1,
    depthLoadOp: "clear",
    depthStoreOp: "store",
    stencilReadOnly: true,
}
```

Attachment options

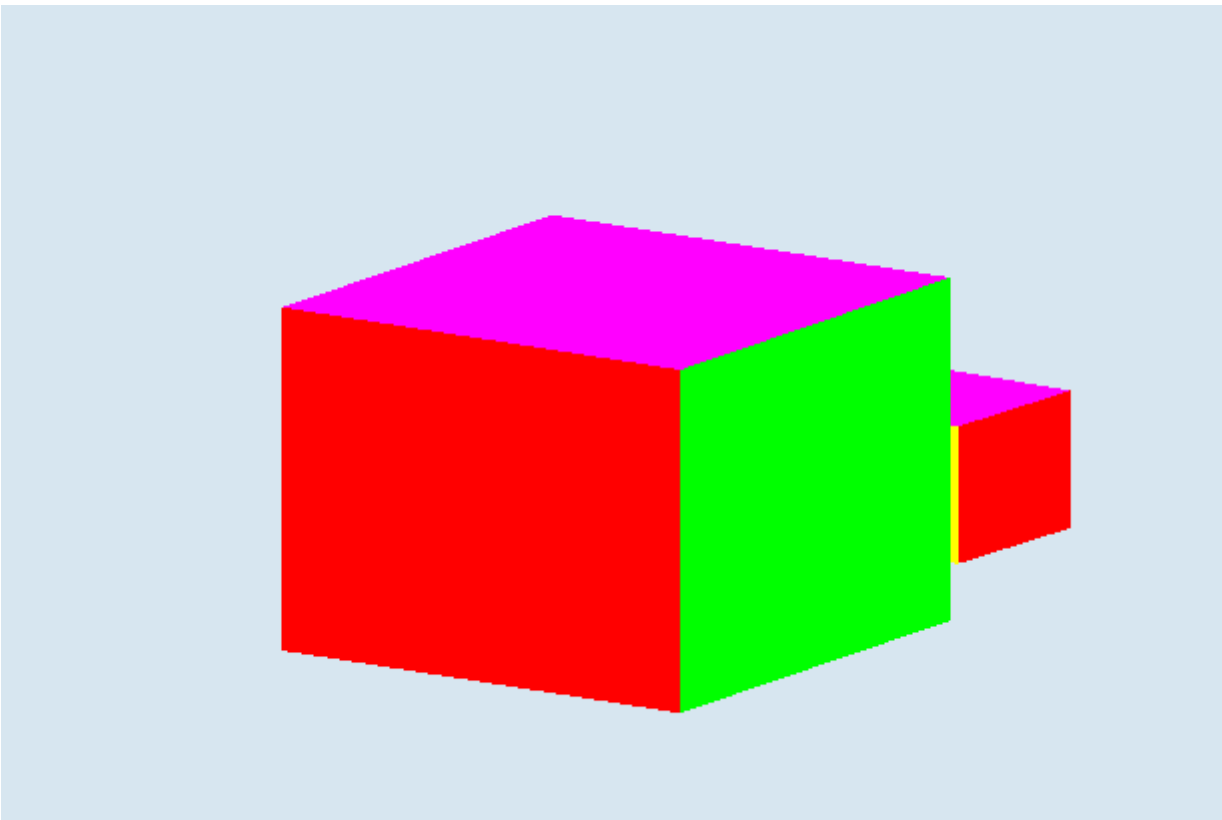
The `view` just points at the depth buffer texture we created.

The `depthClearValue` is the value to reset the depth buffer to when we start a pass. 1 is the farthest value: we set it to 1 so that the first non-culled pixel it draws is always accepted. If we set this to 0, every pixel would be rejected unless it was exactly at z = 0.

`depthLoadOp` defines what happens at the beginning of the pass. We clear the depth buffer to 1. Note: this only happens once per pass, not before each cube is drawn, that would prevent the depth buffer from working.

`depthStoreOp: "store"` just means “don’t discard the value”.

It works!



Two animations

Take a look at `lecture10/screens/with-z-buffer.mp4` to see it working properly. It's a rotating cube in front of a smaller cube that is properly obscured.

I made another video where the color is actually pulling the z-buffer value instead. Take a look at `lecture10/screens/z-values.mp4`. This video is gray scale version of the above video: the brightness of a pixel is its depth value. Brighter means farther away.

What an adventure

We covered a lot!

- We learned about face culling and how it helps us draw solid objects correctly and efficiently
- We saw that it has limitations: it still won't fix objects being drawn in the wrong order if they are at different depths.
- We learned about the depth buffer. This is the key to rendering solid objects in any order.

Make sure to download `sample07`, compile, and run it. Try to disable depth writing to reproduce the failed version without the depth buffer.